false positive. Consider this expectation, where an underlying age_of method is supposed to avoid a specific exception:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { age__of(user) }.not_to raise_error(MissingDataError)
```

This snippet contains a hard-to-spot typo: we wrote age__of with two underscores. When Ruby executes this line, it will raise a NameError, which is *not* a MissingDataError. The expectation will pass, even though our method never even runs!

Because this is such a thorny trap, RSpec 3 will warn you in this situation, and will suggest that you remove the exception class and just call not_to raise_error with no arguments. This form isn't susceptible to the false positive problem, since it will catch *any* exception.

In fact, you can often simplify your specs even further. Since RSpec in effect wraps every example with expect { example.run }.not_to raise_error, you can remove your explicit not_to raise_error check—unless you want to keep it around for clarity.

throw_symbol

Exceptions are designed for, well, exceptional situations, such as an error in program logic. They're not suited for everyday control flow, such as jumping out of a deeply nested loop or method call. For situations like these, Ruby provides the throw construct. You can test your control logic with RSpec's throw_symbol matcher:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { throw :found }.to throw_symbol(:found)
```

Ruby allows you to include an object along with the thrown symbol, and throw_symbol can likewise be used to specify that object via an additional argument:

11-matchers-included-in-rspec-expectations/03/block_matchers.rb
```
expect { throw :found, 10 }.to throw_symbol(:found, a_value > 9)
```

Since throw_symbol is a higher-order matcher, the additional argument can be an exact value, an RSpec matcher, or any object that implements ===.

Yielding

Blocks are one of Ruby's most distinctive features. They allow you to pass around little chunks of code using an easy-to-read syntax. Any method can